

Aula 06 — Colaboração entre objetos

Quando uma operação depende de informações que pertencem a objetos diferentes, quem deve fazer o quê?

TRAJETÓRIA

Hoje vamos investigar

informação repetida



responsabilidades separadas



objetos colaborando

INFORMAÇÃO REPETIDA

Dois itens do mesmo produto

```
ItemPedido itemDaManha =  
    new ItemPedido("Teclado", 150.0, 2);  
  
ItemPedido itemDaTarde =  
    new ItemPedido("Teclado", 150.0, 1);
```

ATIVIDADE

INFORMAÇÃO REPETIDA

Se o preço mudar para 160.0...

```
ItemPedido itemDaManha =  
    new ItemPedido("Teclado", 150.0, 2);  
  
ItemPedido itemDaTarde =  
    new ItemPedido("Teclado", 150.0, 1);
```

Quantos lugares precisam ser encontrados e atualizados?

INFORMAÇÃO REPETIDA

O programa pode terminar assim

Objeto	Descrição	Preço	Quantidade
itemDaManha	Teclado	150.0	2
itemDaTarde	Teclado	160.0	1

Qual é o preço do teclado nesse modelo?

INFORMAÇÃO REPETIDA

O estado mistura duas coisas

Produto

descrição

preço atual do catálogo

Participação no pedido

qual produto

quantidade daquele item

Antes de mover campos, precisamos perguntar o que pertence a quem.

O que pertence a quem?

No Projeto 1:

- um produto existe no catálogo antes de participar de um pedido;
- um item representa certa quantidade desse produto;
- nesta etapa, o item usa o preço atual do catálogo.

1. Quem conhece descrição e preço?
2. Quem conhece a quantidade?
3. O que o item precisa saber para identificar o produto?

RESPONSABILIDADES

Uma distribuição possível

Produto

descricao
preco

ItemPedido

produto
quantidade

ItemPedido precisa conhecer qual **Produto** participa do item.

REGRA DESTA ETAPA

O preço atual pertence ao catálogo

O subtotal acompanha o preço atual mantido por Produto .

Uma compra fechada que preserve o preço praticado exigiria outra decisão de modelagem.

ARMADILHA

RESPONSABILIDADES

“Então a quantidade também vai para Produto”

O mesmo teclado pode aparecer:

- com quantidade 2 em um item;
- com quantidade 1 em outro item.

A quantidade descreve uma participação específica no pedido, não o produto em si.

UMA NOVA NECESSIDADE

Separar resolve a duplicação...

Produto
conhece o preço

precisam
colaborar

ItemPedido
conhece a quantidade

Como calcular o subtotal se nenhum objeto conhece sozinho todos os valores?

O estado que descreve o produto

```
public class Produto {  
    private String descricao;  
    private double preco;  
  
    public Produto(String descricao, double preco) {  
        this.descricao = descricao;  
        if (preco >= 0) {  
            this.preco = preco;  
        }  
    }  
  
    public double getPreco() {  
        return preco;  
    }  
}
```

Produto também pode ser tipo de campo

```
public class ItemPedido {  
    private Produto produto;  
    private int quantidade;  
  
    public ItemPedido(Produto produto, int quantidade) {  
        this.produto = produto;  
        if (quantidade >= 0) {  
            this.quantidade = quantidade;  
        }  
    }  
  
    public Produto getProduto() {  
        return produto;  
    }  
}
```

Um tipo de classe também pode ser tipo de campo

```
private Produto produto;
```

- `Produto` é o tipo;
- `produto` é o nome do campo;
- o campo pode manter uma referência para um objeto `Produto`.

A declaração não executa `new` nem cria outro produto.

O campo permite chegar ao produto

objeto ItemPedido

produto → referência
quantidade = 2

objeto Produto

descricao = "Teclado"
preco = 150.0

O campo não contém uma cópia da descrição e do preço.

CONCEITO-CHAVE

OBJETOS CONECTADOS

Colaboração entre objetos

Objetos colaboram quando um deles solicita a outro uma informação ou um comportamento necessário para cumprir sua própria responsabilidade.

Cada participante contribui com aquilo que conhece ou sabe fazer.

Quem conhece o quê?

Produto

conhece o preço
não conhece a quantidade do item

ItemPedido

conhece a quantidade
sabe qual produto participa

O item pode pedir ao produto a informação que lhe falta.

O cálculo continua no item

```
public double calcularSubtotal() {  
    return produto.getPreco() * quantidade;  
}
```

ItemPedido obtém o preço sem copiar esse valor para seu estado.

LEIA A COLABORAÇÃO

Da esquerda para a direita

produto
acessa a referência



getPreco()
solicita o preço



quantidade
completa o cálculo

Quem deve fazer o quê?

```
Produto teclado = new Produto("Teclado", 150.0);  
ItemPedido item = new ItemPedido(teclado, 3);  
  
double subtotal = item.calcularSubtotal();
```

1. Qual consulta o item faz?
2. Qual valor o produto devolve?
3. Qual estado pertence ao próprio item?
4. Qual subtotal esperamos?

O fluxo completo



O produto fornece seu preço; o item preserva a responsabilidade pelo subtotal.

ATIVIDADE

OBJETOS COMO ARGUMENTOS

Quantos objetos existem?

```
Produto teclado = new Produto("Teclado", 150.0);  
ItemPedido item = new ItemPedido(teclado, 2);
```

1. Quantos objetos do modelo foram criados?
2. Quantas referências chegam ao `Produto` depois da segunda linha?
3. O argumento `teclado` cria uma cópia?

Duas expressões new

new Produto(...)

cria um objeto `Produto`

new ItemPedido(...)

cria um objeto `ItemPedido`

O argumento `teclado` não contém outra expressão `new Produto(...)`.

A referência entra pelo construtor

```
public ItemPedido(Produto produto, int quantidade) {  
    this.produto = produto;  
    // preparação da quantidade  
}
```

teclado
argumento



produto
parâmetro



this.produto
campo

Duas referências, o mesmo produto



A variável e o campo permitem chegar ao mesmo objeto.

Podemos verificar

```
System.out.println(  
    teclado == item.getProduto()  
);
```

resultado true

CONCEITO-CHAVE

OBJETOS COMO ARGUMENTOS

Passar um objeto como argumento

O parâmetro recebe uma referência para o objeto existente.

A passagem não executa `new` nem cria automaticamente uma cópia independente.

ARMADILHA

OBJETOS COMO ARGUMENTOS

“O construtor recebeu um produto, então copiou o objeto”

O construtor guardou a referência recebida:

```
this.produto = produto;
```

Copiar uma referência não é copiar o objeto.

A mesma ideia em uma pousada

Um `Quarto` conhece seu número e o valor da diária. Uma `Reserva` conhece a quantidade de noites para aquele quarto.

```
Quarto quarto = new Quarto(12, 180.0);  
Reserva reserva = new Reserva(quarto, 3);  
  
double total = reserva.calcularTotal();
```

Distribua as responsabilidades

1. Quem conhece o valor da diária?
2. Quem conhece a quantidade de noites?
3. Quem calcula o total da estadia?
4. Qual informação precisa solicitar ao colaborador?
5. Quantos objetos são criados pelo trecho?

O domínio mudou. A colaboração não.

Quarto

número
valor da diária

Reserva

quarto
quantidade de noites
cálculo do total

Reserva solicita ao Quarto a diária necessária para cumprir sua responsabilidade.

PRÓXIMA TENSÃO

O modelo ainda representa um único item

Um pedido real reúne vários objetos `ItemPedido` .

Quem deverá manter esse conjunto e coordenar operações sobre todos os itens?

Ainda não vamos resolver isso

- a colaboração simples entre dois objetos é o núcleo desta aula;
- vários itens exigirão representar e percorrer um conjunto;
- `Pedido` e coleções ficam para a continuidade do Projeto 1.

Uma nova necessidade ficou visível; não precisamos antecipar o mecanismo.

De campos repetidos a objetos que colaboram

cada objeto mantém o estado que lhe pertence



o item guarda uma referência para o produto



o item solicita o preço de que precisa

O que precisamos conseguir explicar

- por que descrição e preço foram separados da quantidade;
- como um campo pode manter referência para outro objeto;
- por que o subtotal continua sendo responsabilidade do item;
- como ler `produto.getPreco()` dentro do cálculo;
- por que passar `teclado` ao construtor não cria uma cópia.

Agora temos uma colaboração simples

Produto

descrição e preço atual

ItemPedido

produto, quantidade e subtotal

Em aberto

um pedido com vários itens